Skill progress monitoring

Catching robot failures while they happen

Pinhas Nehoray Aburmad

Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



Skill progress monitoring
Catching robot failures while they happen

Pinhas Nehoray Aburmad

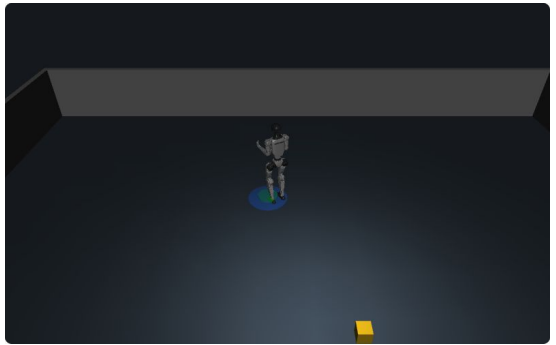
Ben-Gurion University of the Negev
Supervisor: Prof. Ronen Brafman



- Topic: watching a robot while it does a task, and noticing failures *during* the task.
- The talk: the problem, related work, our idea and its math, how we built and tested it, results, what is next.
- About 10 minutes. The live demo is ready if there is time (one command, see slide “What we learned”).

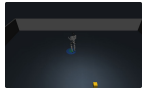
The problem

- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

The problem
- Robots often fail **in the middle** of a task.
- They often **do not notice**.
- They report “done” anyway.
- **Goal:** notice the failure **the moment it happens**.



The robot says “done” at the drop-off (blue). The box (yellow) is still on the floor.

- The picture is a real run from our simulator: the box slipped while carrying; the robot walked on and reported success.
- The robot’s own program only knows what it *commanded*, not what really happened.
- Finding out at the end is late: the time is spent, and some failures cannot be undone.
- So we want an independent watcher that sees the failure when it happens.

Related work 1: monitors

Work	What it does / what is missing
[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]	Checks LTL rules while the system runs: yes / no / not yet. No task stages: “eventually” stays “not yet” to the end.
[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando et al., 2020] [RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]	Monitors that run next to a robot and read its messages. Rules written by hand; no idea of task progress.
[Runtime Verification with State Estimation, Stoller et al., 2011]	Guesses a hidden state with a probability model. Needs that model; no task stages.

Related work 1: monitors
Work
What it does / what is missing
[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]
Checks LTL rules while the system runs: yes / no / not yet.
No task stages: “eventually” stays “not yet” to the end.
[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando et al., 2020]
[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]
Monitors that run next to a robot and read its messages.
Rules written by hand; no idea of task progress.
[Runtime Verification with State Estimation, Stoller et al., 2011]
Guesses a hidden state with a probability model.
Needs that model; no task stages.

- **Bauer et al. 2011 (TOSEM):** the standard theory of LTL monitoring (“LTL_3”): after each step, true, false or “?”; the smallest monitor that answers as early as possible. We build on it. Gap: “eventually deliver” stays “?” to the end; no stages.
- **ROSMonitoring 2020 (TAROS):** turns ROS topics into monitor nodes; tested on a simulated Mars rover. **RTAMT 2024 (STTT):** Signal Temporal Logic monitors that say *how much* a rule holds; used with ROS on a Toyota robot. Gap: rules written by hand; no task progress.
- **Stoller et al. 2011 (RV):** when events are missed, a Hidden Markov Model fills the gaps and gives the probability that a rule holds. Closest to our POMDP view. Gap: needs a probability model; no stages.

Related work 2: from words, from data

Work	What it does / what is missing
[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu et al., 2023] [nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs, Cosler et al., 2023] [NL2TL: Transforming Natural Languages to Temporal Logics using LLMs, Chen et al., 2023]	Turn English sentences into LTL rules. Used to plan or to write specs, not to watch a running task.
[FailSafe: Reasoning and Recovery from Failures in VLA Models, Lin et al., 2025] [Foresight: Failure Detection for Long-Horizon Robotic Manipulation, Zhang et al., 2026] [Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor et al., 2026]	Learn to spot failures from camera images. Need training data; hard to say why an alarm fired.

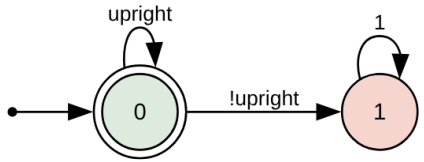
Our gap: rules made from plain words + task stages + only the robot's own sensors.

Related work 2: from words, from data
Work
What it does / what is missing
[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu et al., 2023]
[nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with LLMs, Cosler et al., 2023]
[NL2TL: Transforming Natural Languages to Temporal Logics using LLMs, Chen et al., 2023]
Turn English sentences into LTL rules.
Used to plan or to write specs, not to watch a running task.
[FailSafe: Reasoning and Recovery from Failures in VLA Models, Lin et al., 2025]
[Foresight: Failure Detection for Long-Horizon Robotic Manipulation, Zhang et al., 2026]
[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor et al., 2026]
Learn to spot failures from camera images.
Need training data; hard to say why an alarm fired.
Our gap: rules made from plain words + task stages + only the robot's own sensors.

- **Lang2LTL (CoRL 2023):** English route commands to LTL (GPT-4 finds places, a small model writes the formula); works in new cities. For planning.
- **nl2spec (CAV 2023):** an LLM translates a sentence to LTL and shows which words became which part, so a person can fix it: 16/36 hard cases alone, 31/36 with a human. For writing specs.
- **NL2TL (EMNLP 2023):** hides the nouns, then a fine-tuned T5 writes the logic; about 28k training pairs.
- **FailSafe (2025):** trains a vision-language model on failures made in simulation; up to +22.6 points success for one policy. **Foresight (2026):** predicts the near future from camera and planned actions; alarm when it looks wrong. **Embedding TL (2026):** logic over image embeddings; the authors say its facts are not yet human-readable.
- Gap: none makes the monitor from the task description, with stages, from the robot's own sensors.

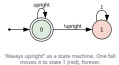
Background: what is a monitor?

- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.



"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

Background: what is a monitor?
- A program that **watches** the robot's messages.
- It checks **rules over time**, written in **LTL** (a logic of time).
- Example: "*always* stay upright".
- Each rule becomes a small **state machine**.
- Problem: "*eventually* deliver" **never fails early**.

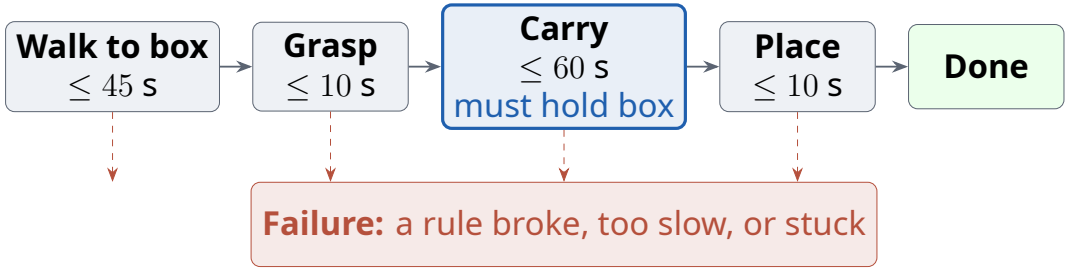


"Always upright" as a state machine. One fall moves it to state 1 (red), forever.

- LTL = Linear Temporal Logic: rules with "always" (G) and "eventually" (F).
- The tool Spot turns each rule into a small automaton. Every second the monitor feeds it the facts it measured (upright? holding the box?) and takes one step.
- "Always upright" fails the moment the robot falls: easy.
- "Eventually deliver" can never fail early, because delivery might still happen later. This is the weakness we fix next.

Our idea: the stages of the task

Each stage has its own rules and its own time limit.



While carrying, the robot **must hold the box**. A drop is caught **at once**.

Our idea: the stages of the task
Each stage has its own rules and its own time limit.
Walk to box ≤ 45 s
Grasp ≤ 10 s
Carry ≤ 60 s must hold box
Place ≤ 10 s
Done
Failure: a rule broke, too slow, or stuck
While carrying, the robot must hold the box. A drop is caught at once.

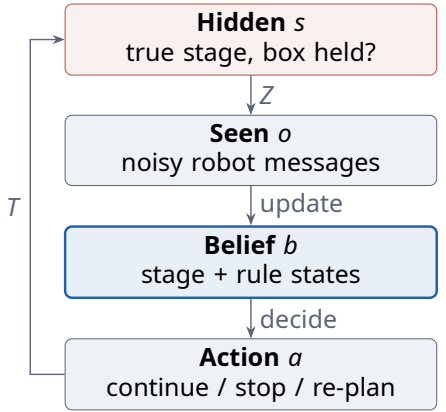
- We call these stages “progress states”. Each stage says: what must stay true, what ends the stage, a time limit, and that the robot must keep making progress.
- These stages were written by the LLM from the plain description of “fetch and deliver”.
- Key example: in Carry, “holding the box” must stay true. A drop breaks it in the next second.
- The same fact (“not holding”) is normal while walking to the box. A rule without stages cannot say “must hold”, because it would be false at the start.

The mathematical model: a POMDP

- **Hidden** s: what really happens (stage, box held?).
- **Seen** o: the robot's messages.
- **Belief** b: our best guess of s.
- **Action** a: continue, stop, or re-plan.

$$b'(s') \;\propto\; \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} \; b(s)$$

Today: the guess is one point (stage + rule states); the update is one rule step.

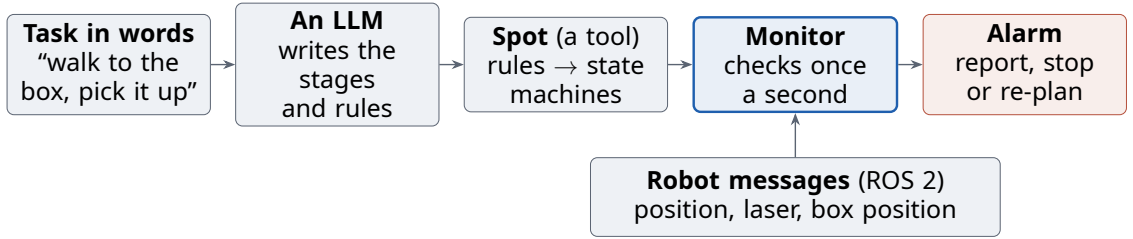


[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]

The mathematical model: a POMDP
Hidden s: what really happens (stage, box held?).
Seen o: the robot's messages.
Belief b: our best guess of s.
Action a: continue, stop, or re-plan.
$b'(s') \propto \underbrace{Z(o \mid s')}_{\text{does } o \text{ fit } s'?} \sum_{s} \underbrace{T(s' \mid s, a)}_{\text{how } s \text{ changes}} b(s)$
Today: the guess is one point (stage + rule states); the update is one rule step.
Hidden s true stage, box held?
Z
Seen o noisy robot messages
update
Belief b stage + rule states
decide
Action a continue / stop / re-plan
T
[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]

- POMDP (Kaelbling et al., AI Journal 1998): a hidden state, noisy observations, a belief, and actions chosen from the belief.
- The formula, right to left: move the old guess forward with how the world changes (T), then weigh each state by how well it explains what we saw (Z).
- Why stages help: the stage is part of s. "Not holding" fits "walking" but not "carrying", so in Carry one observation shows the failure.
- Honest: today the guess is one point, the update one automaton step, the action a fixed rule. No probabilities yet (last step of "To be done").
- Value of information: early knowledge helps only if it changes the outcome; tested with deadlines in Experiment 2.

Method: how the monitor is built and run



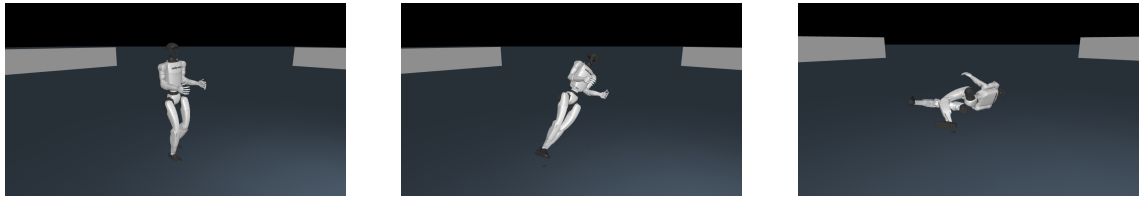
- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

Method: how the monitor is built and run
Task in words “walk to the box, pick it up” → An LLM writes the stages and rules → Spot (a tool) rules → state machines → Monitor checks once a second → Alarm report, stop or re-plan
Robot messages (ROS 2) position, laser, box position
- The rules are written **automatically**, from the task in words.
- The monitor uses **only sensors**. It never asks the planner.

- The LLM (Claude) wrote the whole spec in one try; automatic checks made sure it loads and that every fact it uses can be measured. It was saved before any experiment ran.
- An “evaluator” turns the raw messages into true/false facts each second, e.g. “holding the box” is inferred from where the box is relative to the robot.
- Evaluator and monitor are separate ROS 2 programs, exactly as on a real robot.
- The monitor never reads the planner’s own status: a test enforces this. (Experiment 1 shows why.)

Test bed: a humanoid robot in a simulator

- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.



Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

Test bed: a humanoid robot in a simulator
- Unitree **G1** robot in the **MuJoCo** physics simulator.
- We **cause failures on purpose**: push, stall, drop, failed grasp.
- We compare the monitor **with stages** and **without**.

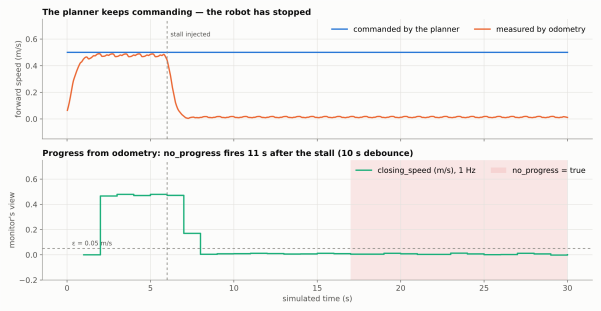


Walking → pushed → fallen. The simulator knows the truth; the monitor does not.

- The G1 walks with Unitree's own released walking controller.
- Each failure is injected at a known time, so we can measure how fast it is caught.
- "Without stages" = the same spec with all stages merged into one (a fair control: same facts, same safety rules).
- Success or failure is judged by the simulator's ground truth, which the monitor never sees.

Experiment 1: a short walk (1800 runs)

- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors**.
- Without that: only **1 of 15** caught.



The planner still says "go". The robot has stopped. The monitor sees no progress.

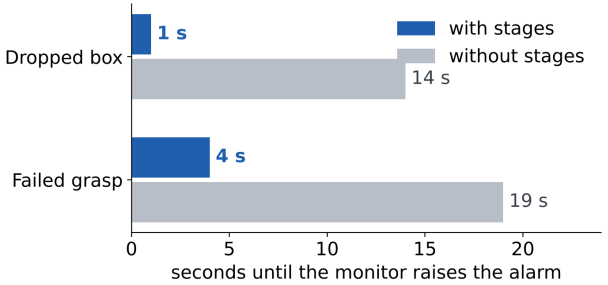
- 10 failure types × 20 seeds × 3 versions × 3 reactions = 1800 runs.
- Walking has two stages that check the same things, so stages add little: falls are caught in 1–7 s by every version.
- Stalls are the difference. "No progress" is measured from odometry against the goal; without it only 1 of 15 stalls is caught.
- The figure: the planner keeps commanding 0.5 m/s; the robot has stopped. A monitor that trusted the planner would see nothing wrong.

Experiment 1: a short walk (1800 runs)
- Stages helped **little** here: 15 vs. 14 of 15 stalls caught.
- What mattered: measure progress **from the robot's sensors.**
- Without that: only **1 of 15** caught.



Experiment 2: fetch and deliver (3360 runs)

- Four stages.
- Box dropped: caught after **1 s** with stages, 14 s without.
- Grasp failed: **4 s** vs. 19 s.
- Earlier in **all 20** runs.



Middle value (median) of 20 runs for each failure.

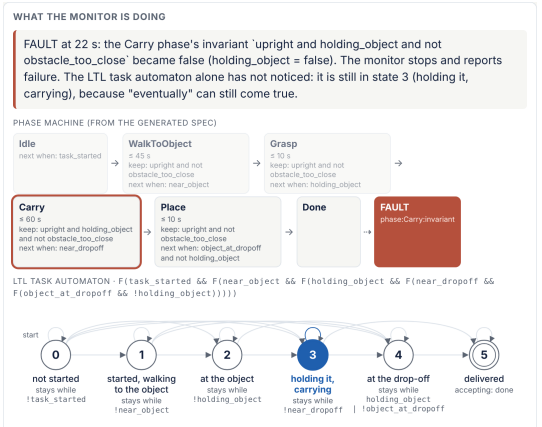
Experiment 2: fetch and deliver (3360 runs)
• Four stages.
• Box dropped: caught after 1 s with stages, 14 s without.
• Grasp failed: 4 s vs. 19 s.
• Earlier in all 20 runs.
Middle value (median) of 20 runs for each failure.

- Walk to a box, grasp it, carry it 2.5–4 m, place it. Evaluator and monitor ran as separate ROS 2 programs; the box position came from a noisy detector (2 cm noise, 5% missed).
- Drop: caught 1 s after it happens, because Carry requires holding. Without stages the monitor only notices when the robot claims success, 14 s later.
- Failed grasp: caught by the Grasp stage's time limit (4 s vs. 19 s).
- Paired by scenario: earlier in 20 of 20 runs, later in none (sign test $p = 1.9 \cdot 10^{-6}$).

One run, second by second

- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

The stage caught it.



From the live demo (one run). The current stage and state are lit.

One run, second by second
- **21.6 s:** the box drops.
- **22 s:** the Carry rule breaks. Alarm.
- The plain LTL rule still waits: “maybe later”.

The stage caught it.

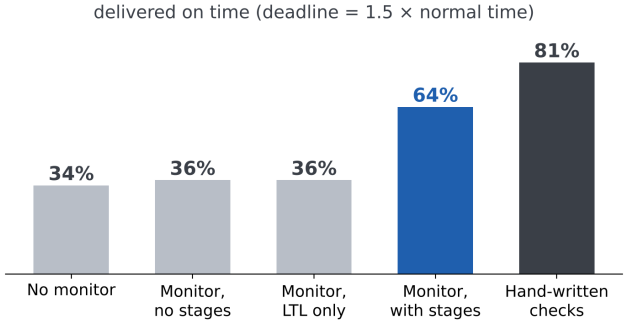


From the live demo (one run). The current stage and state are lit.

- This is one run from the live demo page (seed 0), not a statistic.
- Top: the stages, Carry outlined in red. Bottom: the 6-state LTL automaton for “walk, grasp, carry, place, in order”.
- The automaton is still in state 3 (“holding, carrying”): it has no “failed” state, because “eventually” can always still come true.
- This is the whole idea in one picture: the stage catches what the LTL rule alone cannot.

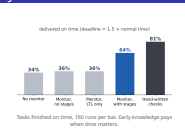
Does catching early matter?

- With no time limit: all monitors ≈ 82–84%.
- With a time limit: **64%** with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



Tasks finished on time, 160 runs per bar. Early knowledge pays when time matters.

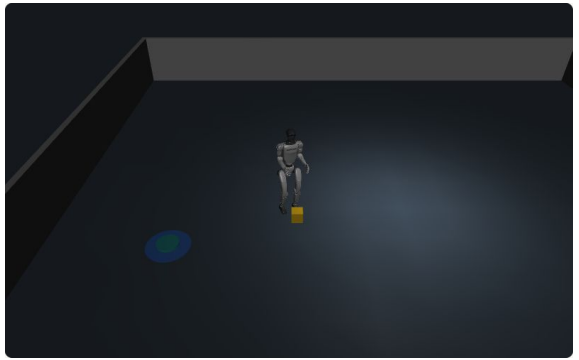
Does catching early matter?
- With no time limit: all monitors ≈ 82–84%.
- With a time limit: 64% with stages, 36% without.
- Hand-written checks: 81%, but someone must **predict every failure**.



- On an alarm the robot re-plans. With no deadline, every monitor ends at 82–84% success, because a late re-plan still works.
- With a deadline of 1.5× the normal time: 64% with stages vs. 36% without ($p = 7 \cdot 10^{-7}$). LTL alone (no stages): 36%.
- This is the “value of information” from the POMDP slide: early knowledge pays only when time matters.
- Hand-written checks inside the robot program do better (81%), because the program knows its own state. But an engineer had to foresee each failure: see the next slide.

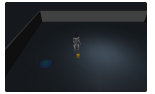
Failures nobody planned for

- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: **40 of 40** missed.
- With our monitor: **0 of 40** missed.
- Missed by all: too slow, or a long detour.



The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

Failures nobody planned for
- Box left at the wrong place; robot too close to a wall.
- Hand-written checks: 40 of 40 missed.
- With our monitor: 0 of 40 missed.
- Missed by all: too slow, or a long detour.

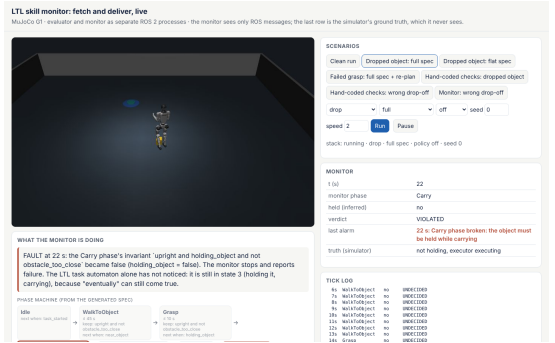


The box is far from the drop-off (blue), yet the robot says “done”. Our monitor catches it.

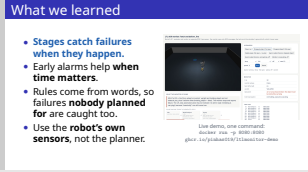
- New failures, one per requirement of the task description that the hand-written checks do not cover. The design and predictions were written down before the run.
- Wrong drop-off: the robot’s own idea of the drop-off is wrong, so its own check passes and it reports success. Too close: the navigator grazes obstacles.
- Hand-written checks alone: 40 of 40 failed silently. Adding our monitor: 0 of 40, at no measurable cost elsewhere.
- Missed by everyone: running slowly, or a detour, that stay inside the time limits. Our prediction for “slow” was wrong, and we say so.

What we learned

- **Stages catch failures when they happen.**
- Early alarms help **when time matters**.
- Rules come from words, so failures **nobody planned for** are caught too.
- Use the **robot's own sensors**, not the planner.

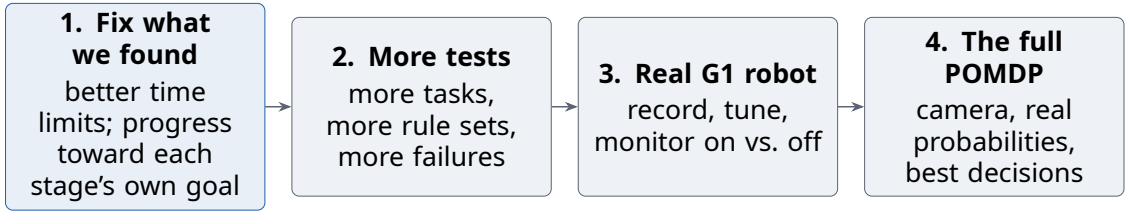


Live demo, one command:
`docker run -p 8080:8080 ghcr.io/pinhas019/ltlmonitor-demo`



- Four take-aways, one per line on the slide.
- If there is time, show the demo: open http://localhost:8080, click "Dropped object: full spec", press Pause at 22 s.
- The demo page draws the stages and the automaton live, from the monitor's own ROS 2 messages.

To be done



Limits today: the grasp is simulated (no arms on this G1), the camera is simulated, the thresholds are not tuned.

To be done
1. Fix what we found
better time limits; progress toward each stage's own goal
2. More tests
more tasks, more rule sets, more failures
3. Real G1 robot
record, tune, monitor on vs. off
4. The full POMDP
camera, real probabilities, best decisions
Limits today: the grasp is simulated (no arms on this G1), the camera is simulated, the thresholds are not tuned.

- 1: time limits from normal runs; progress toward each stage's own target (would catch a skipped pick-up); after re-plan, restart at the current stage.
- 2: several generated specs per task; more tasks and failures.
- 3: real G1: record, tune thresholds, monitor on vs. off.
- 4: full POMDP (Paper 2): camera facts with probabilities, a real belief, stop / continue / re-plan by expected cost.
- Limits: the legs-only G1 cannot grasp (the box is attached); the box position is from a simulated detector.

References I

[Runtime Verification for LTL and TLTL, Bauer, Leucker, Schallhart, 2011]. ACM TOSEM 20.

[NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models, Chen, Gandhi, Zhang, Fan, 2023]. EMNLP.

[nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models, Cosler, Hahn, Mendoza, et al., 2023]. CAV.

[ROSMonitoring: A Runtime Verification Framework for ROS, Ferrando, Cardoso, Fisher, Ancona, et al., 2020]. TAROS.

[Planning and Acting in Partially Observable Stochastic Domains, Kaelbling, Littman, Cassandra, 1998]. Artificial Intelligence 101.

[Runtime Monitoring of Perception-Based Autonomous Systems via Embedding Temporal Logic, Kapoor, Hammer, Kapoor, et al., 2026]. arXiv.

[FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models, Lin, Duan, Fang, Fox, et al., 2025]. arXiv.

[Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments, Liu, Yang, Idrees, Liang, et al., 2023]. CoRL.

[Runtime Verification with State Estimation, Stoller, Bartocci, Seyster, Grosu, Havelund, Smolka, Zadok, 2011]. RV, LNCS 7186.

References I

Every entry is a link. Venues and authors come from the verified BibTeX entries in `papers/`; FailSafe, Foresight and Embedding-TL have no verified venue yet, so they link to arXiv.

References II

[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]. STTT 26.

[Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents, Zhang, Lu, Wang, Kang, et al., 2026]. arXiv.

References II
[RTAMT: Runtime Robustness Monitors with Application to CPS and Robotics, Yamaguchi, Hoxha, Ničković, 2024]. STTT 26.
[Foresight: Failure Detection for Long-Horizon Robotic Manipulation with Action-Conditioned World Model Latents, Zhang, Lu, Wang, Kang, et al., 2026]. arXiv.